DESIGN AND ANANYSIS ALGORITHMS

LECTURE 4

Divide and Conquer Method

Review the Previous Lesson

- Problem can be solve by recursion.
- Format of a recursive algorithm
- Analysis of recursive algorithm
  - Correctness: proof by inductive
  - Time complexity: deriving and solving recurrence relations
- Recursion Elimination

Lecture outline

- Introduction
  - Divide and conquer diagram
  - Analysis of divide and conquer algorithm
- Examples
- Other types of simplify and conquer
- Exercises

Reference link and book chapter

Prof. Xukai Zou, Computer Science Dept.,

Indiana University Purdue University Indianapolis

http://cs.iupui.edu/~xzou/

Anany's book, chapter 5

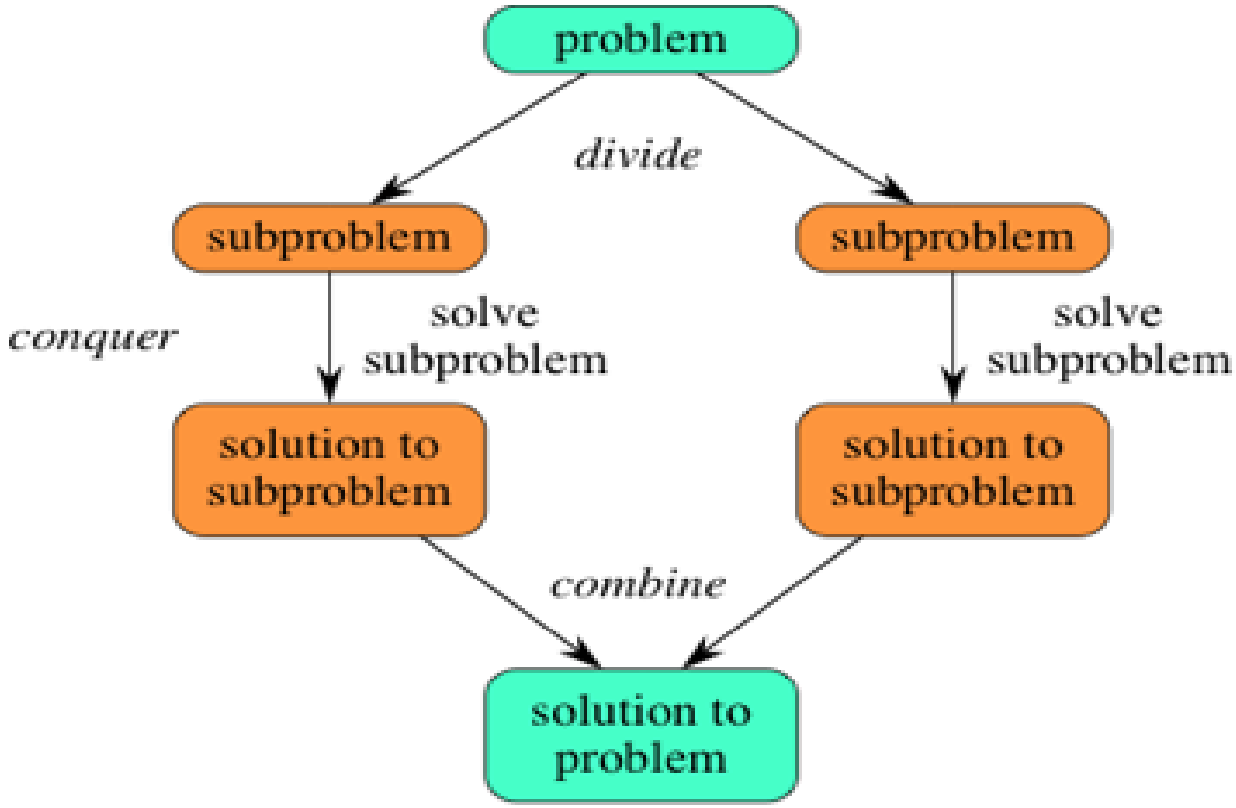
Introduction

- ✓ Divide and conquer strategy
- ✓ Divide and conquer visualization

Divide and Conquer Strategy

- Divide and Conquer (D&C) is an important algorithm design technique based on recursion.
- The D & C strategy solves a problem by:
  1. ***Divide*** (chia) or “breaking” or “partitioning” the problem into subproblems of the same type.
  2. ***Conquer*** (trị) the subproblems by solving them recursively.
  3. ***Combine*** the solutions to the subproblems into the solution for the original given problem.

Divide and Conquer Visualization



Divide or “breaking” or “partitioning” on the input data until each part small enough to get the solution of subproblem (recursive base case).

D&C Algorithm Diagram

- Consider D&C(R) is algorithm of problem P with input range R.
- If R can be partitioned into *n* sub-ranges ($R= R_1 \cup R_2 \cup \ldots \cup R_n$) and D&C($R_0$) has solution
- Diagram:

```
D&C(R)≡
        if(R=R0)
            Get solution D&C(R0);
        else
            Partition R into R1,R2,…,Rn
            for(i=1…n)
                D&C(Ri)
            Combine the solutions.
End.
```

D&C Algorithm Correctness

- Prove D&C algorithm correctness by induction:
  - Prove by induction on the "input range" of the problem.
  - ***Base case:*** solution D&C(R_0) is correct.
  - ***Inductive assumption***: Assume that D&C(R) has solution with input range R such |R|=*n.*
  - ***General case****:* Proof D&C(R_1) works correctly with input range R_1 such $|R_1|=n+1$.

D&C Algorithm Time Complexity

- Deriving recurrence relations:

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ aT(n/b) + n^d & \text{if } n > 1, a \geq 1, b > 1, d \geq 0 \end{cases}$$

- Solving recurrence relations:
  - Use master theorem

$$T(n) = \begin{cases} O(n^d) & \text{if } a < b^d \\ O(n^d \log n) & \text{if } a = b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

D&C applications

- Sorting: Mergesort and Quicksort
- Tree traversals
- Binary search
- Matrix multiplication - Strassen’s algorithm
- Closest Pair of Points
- Convex hull - QuickHull algorithm
- …

Examples

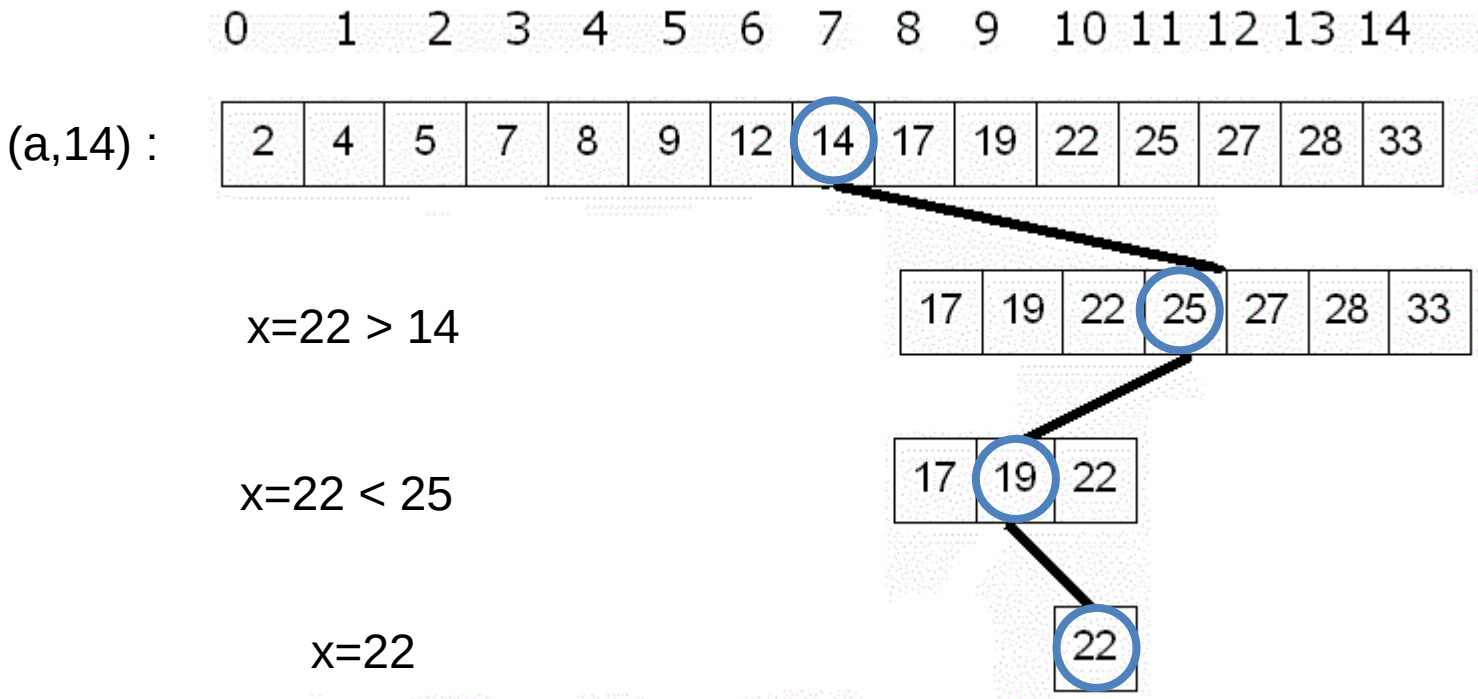
- ✓ Binary Search
- ✓ MinMax
- ✓ Strassen's Matrix Multiplication

Binary Search

- Binary search in sorted list problem:
  - Input: list (a_1,…a_n), x;
  - Output: position i such a_i=x, i = -1 if otherwise.
- Idea:
  - Compare x with item at the middle of the list
  - Use the compare result to decide continue search in which part

Binary Search

- Binary search illustration:



Binary Search

- Binary search function diagram:

```
BinarySearch(a,x, L,R):int ≡
    if (L=R) return (x=a_L ? L : -1)
    else
        M = (L+R)/2;
        if (x = a_M) return (M);
        else
            if (x<a_M) BinarySearch(a,x,L,M)
            else BinarySearch(a,x,M+1,R)
        endif;
    endif;
End.
```

Binary Search

- Binary search main program:

```
main() ≡
     Input (a,n);
     Input x;
     result = BinarySearch(a,x,0,n-1);
     Output result;
End.
```

Binary Search

- Prove the correctness: induction by size of the list *n*
  - *Base case*: $n = R\text{-}L+1 = 1$ (only 1 element)
    - Statement `return (x=a`$_L$ `? L : -1)` return `L` or `-1`
  - *Inductive assumption*: Algorith correct with list has $n \le R\text{-}L+1$
    - Mean `BinarySearch(a,x, L,R)` return the correct position of *x* with all list has n' elements, such $1 \le n' \le n = R\text{-}L+1$
  - *General case*: Prove algorithm correct with list has $n+1 = R\text{-}L+2$
    - $M=(L+R+1)/2;\ L \le M \le R$
    - If $x=a_M$ the output is M : correct
    - If $x<a_M$ the output is output of $x \in a_L..a_M$? By inductive assumption we have `BinarySearch(a,x,L,M)` is correct, since $1 \le M\text{-}L+1=(R\text{-}L+1)/2+1 \le R\text{-}L+1$
    - If $x>a_M$ same situation. ■

Binary Search

- Time running complexity

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(n/2) + 1 & \text{if } n > 1 \end{cases}$$

$$\Rightarrow \mathrm{T}(n) = \mathrm{O}(\log n)$$

- Recursion Elimination
  - Khử đệ quy sử dụng lược đồ đệ quy dạng ED[AP]

MinMax

- MinMax problem: Find the maximum and minimum elements in a list . How many comparisons are needed?
  - Input: list (a_1,…a_n);
  - Output: *min* and *max* value of the list.
- Idea:
  - Naive algorithm: Use two independent loop: one find *min*, the other find *max*. Time complexity O(n); Total comparisons 2(*n-1).*
  - Divide and conquer: Divide the list in half. Find the maximum and minimum in each half recursively, return max and min of two results

MinMax D&C

- Algorithm diagram:

```
MinMax(a,L,R):(int,int) ≡
      if (R-L<=1)
          return(min(a_L,a_R),max(a_L,a_R));
      else
          (min1,max1) = MinMax(a,L,(L+R)/2);
          (min2,max2) = MinMax(a,(L+R)/2+1,R);
          return(min(min1,min2),max(max1,max2));
      endif;
End.
```

The first call with list has *n* elements: `MinMax(a,0,n-1)`

MinMax D&C

- Prove the correctness:
  - Induction by size of the list $n = R\text{-}L+1$
  - *Base case*: $n = R\text{-}L+1 \leq 2$ (list has 1 or 2 elements)
  - *Inductive assumption*: Algorithm correct with all list has $n = R\text{-}L+1$ ($n>2$)
    - Mean `MinMax(a,L,R)` return the correct *min*, *max* values in interval [L..R] (from a_L to a_R)
  - *General case*: Prove the algorithm correct with $n+1 = R\text{-}L+2$
    - Apply induction assumption to each recursive call.
    - `MinMax(a,L,(L+R)/2);`
    - `MinMax(a, (L+R)/2+1, R);`

MinMax D&C

- Time running complexity

$$T(n) = \begin{cases} 1 \text{ if } n \leq 2 \\ 2T(n/2) + 2 \text{ if } n > 1 \end{cases}$$

$$\Rightarrow \mathrm{T}(n) = \mathrm{O}(?)$$

Compare with naive algorithm with two loop?

- The comparisons (assume $n = 2^m$)

$$\mathrm{T}(n) = \mathrm{T}(2^m) = 2\mathrm{T}(2^{m-1}) + 2 = 2(2\mathrm{T}(2^{m-2}) + 2) + 2^1 = 2^2\mathrm{T}(2^{m-2}) + 2^2 + 2^1$$

$$= \ldots = 2^{m-1}\mathrm{T}(2^{m-(m-1)}) + 2^{m-2} + \ldots + 2^2 + 2^1 = 2^{m-1} + \ldots + 2^2 + 2^1$$

$$= 2^m - 2 = n - 2$$

The comparison is about half versus naïve algorithm.

Matrix Multiplication

- Matrix multiplication: $X = Y \times Z$
  - Input: Two matrices Y, Z size $n \times n$
  - Output: X size $n \times n$
- Idea:
  - Naive algorithm: three nested loops.
  - Divide and conquer: Divide X, Y, Z each into four $(n/2) \times (n/2)$ matrices

$$X = \begin{bmatrix} I & J \\ K & L \end{bmatrix}$$

$$Y = \begin{bmatrix} A & B \\ C & D \end{bmatrix}$$

$$Z = \begin{bmatrix} E & F \\ G & H \end{bmatrix}$$

then

$$\begin{aligned} I &= AE + BG \\ J &= AF + BH \\ K &= CE + DG \\ L &= CF + DH \end{aligned}$$

D&C Matrix Multiplication

- Divide and conquer matrix multiplication:
  - Divide *X*, *Y*, *Z* each into four (*n*/2) x (*n*/2) matrices

$$X = \begin{bmatrix} I & J \\ K & L \end{bmatrix} \qquad Y = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \qquad Z = \begin{bmatrix} E & F \\ G & H \end{bmatrix}$$

then

$$\begin{aligned} I &= AE + BG \\ J &= AF + BH \\ K &= CE + DG \\ L &= CF + DH \end{aligned}$$

  - Time complexity

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 8T(n/2) + n^2 & \text{if } n > 1 \end{cases}$$

Strassen's Matrix Multiplication

- Stranssen's matrix multiplication:
  - Improve divide and conquer

Set:

$$
\begin{aligned}
M_1 &:= (A+C)(E+F) \\
M_2 &:= (B+D)(G+H) \\
M_3 &:= (A-D)(E+H) \\
M_4 &:= A(F-H) \\
M_5 &:= (C+D)E \\
M_6 &:= (A+B)H \\
M_7 &:= D(G-E)
\end{aligned}
$$

then

$$
\begin{aligned}
I &= AE+BG \\
J &= AF+BH \\
K &= CE+DG \\
L &= CF+DH
\end{aligned}
$$

$$
\begin{aligned}
I &:= M_2+M_3-M_6-M_7 \\
J &:= M_4+M_6 \\
K &:= M_5+M_7 \\
L &:= M_1-M_3-M_4-M_5
\end{aligned}
$$

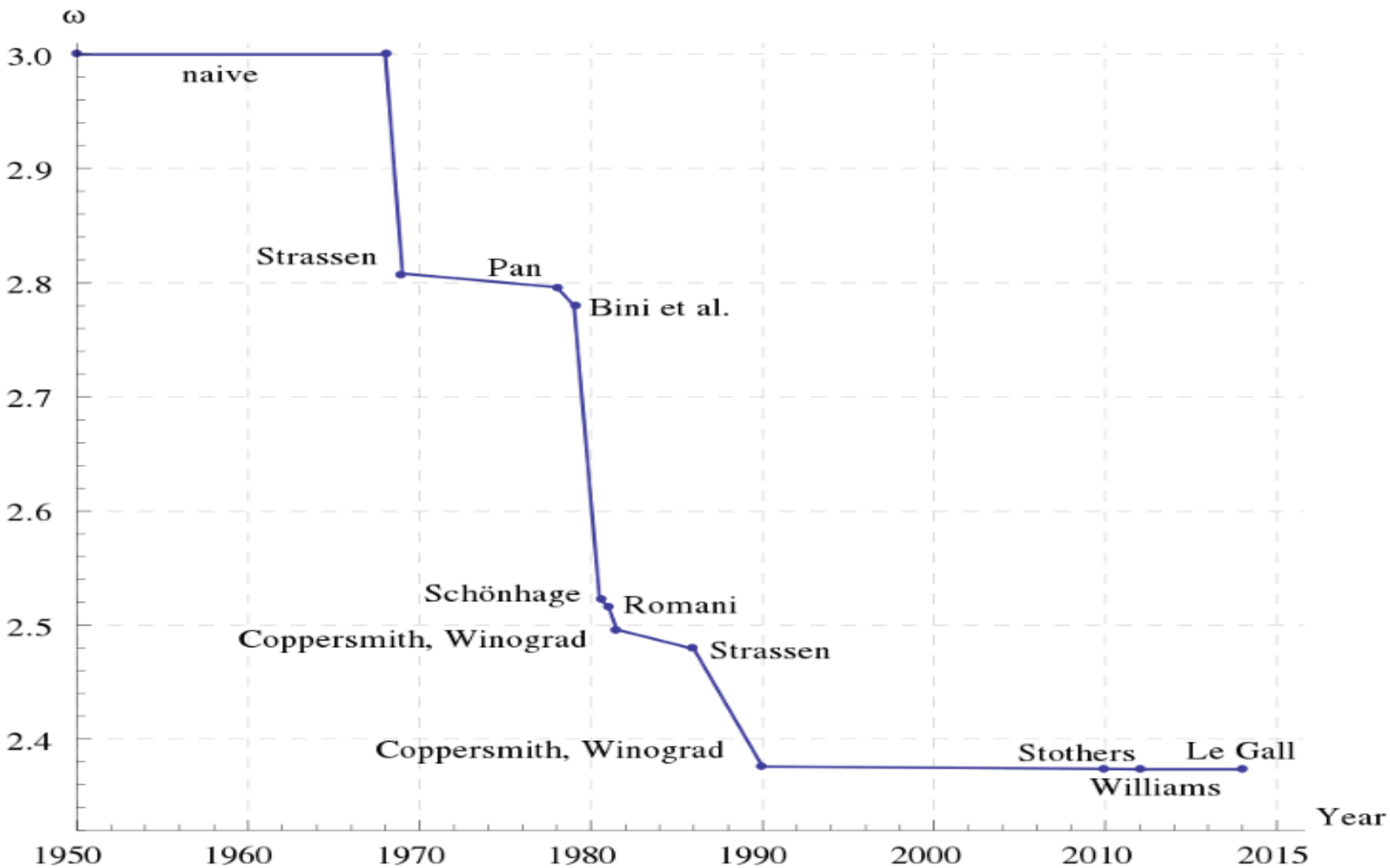
D&C Matrix Multiplication

- Divide and conquer matrix multiplication:
  - Time complexity

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 7T(n/2) + n^2 & \text{if } n > 1 \end{cases}$$

$$\Rightarrow \mathrm{T}(n) = \mathrm{O}(?)$$

D&C Matrix Multiplication



Những cải tiến về ω trong 5 thập kỷ qua. [Nguồn: Wikipedia]

Simplify and Conquer

- ✓ Simplify and Conquer strategy
- ✓ Decrease and Conquer
- ✓ Transform and Conquer

Simplify and Conquer Strategy

- The Simplify and Conquer strategy
  - ***Simplify*** the problem (làm đơn giản) .
  - ***Conquer*** the simpler subproblems (trị).
- Three ways to Simplify
  - Divide ⇨ Divide and Conquer
  - Decrease ⇨ Decrease and Conquer
  - Transform ⇨ Transform and Conquer

Decrease and Conquer

- The decrease-and-conquer technique is based on the relationship between a solution of a problem and a solution to the similar smaller problem(s).
- Three ways variations of decrease-and-conquer
  - Decrease by a constant:
    - Ex: (n-1) in factorial Algorithm, MinMax...
  - Decrease by a constant factor
    - Ex: Binary Search; Fake-Coin Problem, Josephus Problem
  - Variable size decrease
    - Ex: Selection problem, the Game of Nim

(Anany's book, sec 4.4, 4.5)

Transform and Conquer

- The transform-and-conquer technique has two-stage: Transformation stage (the problem is modified into more amenable to solution); and conquering stage, it is solved.
- Three ways variations of tranform-and-conquer
  - *Simplification (đơn giản hóa):* Transformation to a simpler problem.
  - *Representation change (đổi biểu diễn)*: Transformation to another representation of the same problem.
  - *Problem reduction (suy giảm bài toán)*:Transformation another problem for which be sovled.

(Anany's book, chapter 6)

Exercises

- Program the designed algorithms
- Other examples or application of
  - Divide and conquer
  - Decrease and conquer
  - Transform and conquer
- More detail in Hw4_DivideAndConquer.doc